Object Oriented Programming (CSEG1044)

Unit 05 – Lecture 03: Wrapped Collections + ‘Collections‘ Utilities + Wrappers

April 1, 2026

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Collections Framework and Wrapper Classes

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Collections Framework and Wrapper Classes
- **Course thread:** Use Wrapped Collections + ‘Collections‘ Utilities + Wrappers to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Recall Before You Start

- **Collection interfaces**: Utilities and wrappers only make sense once List, Set, and Map roles are already clear. Main reference: `Unit 05 / Lecture 01 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Recall Before You Start

- **Collection interfaces**: Utilities and wrappers only make sense once List, Set, and Map roles are already clear. Main reference: `Unit 05 / Lecture 01 slides/notes`.
- **Primitive and object types**: Wrapper classes connect directly to the type-conversion ideas from Unit 01. Main reference: `Unit 01 / Lecture 03 slides/notes`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Learning Outcomes

- Use common `Collections` utilities (sort, min/max, shuffle, reverse).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Learning Outcomes

- Use common `Collections` utilities (sort, min/max, shuffle, reverse).
- Use wrapped collections (unmodifiable/synchronized/checked) appropriately.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Learning Outcomes

- Use common `Collections` utilities (sort, min/max, shuffle, reverse).
- Use wrapped collections (unmodifiable/synchronized/checked) appropriately.
- Avoid common wrapper/autoboxing pitfalls (and know class loading basics).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Terms to Know

- **Utility class**: class with static helper methods (e.g., `Collections`).
- **Unmodifiable view**: wrapper that blocks structural modification.
- **Synchronized wrapper**: wrapper that adds basic synchronization around operations.
- **Checked wrapper**: wrapper that enforces runtime type checks on adds.
- **Wrapper class**: object form of a primitive (e.g., `Integer` for `int`).
- **Autoboxing/unboxing**: automatic conversion between primitive and wrapper.
- **Class loading**: JVM loads class bytecode into memory when needed.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).;

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *`Collections` utilities (common ones)* → *Wrapped collections (why/when)* → *Wrapper classes + autoboxing pitfalls* → *Class loading basics (intro)*.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *`Collections` utilities (common ones)* → *Wrapped collections (why/when)* → *Wrapper classes + autoboxing pitfalls* → *Class loading basics (intro)*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *`Collections` utilities (common ones)* → *Wrapped collections (why/when)* → *Wrapper classes + autoboxing pitfalls* → *Class loading basics (intro)*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Unit 05: Collections and Wrapper Class

- Lecture focus: Wrapped Collections + ‘Collections‘ Utilities + Wrappers

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Today: Roadmap

- ‘Collections‘ utilities: sort, min/max, shuffle (overview)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Today: Roadmap

- ‘Collections‘ utilities: sort, min/max, shuffle (overview)
- Wrapped collections: unmodifiable/synchronized/checked (why/when)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Today: Roadmap

- ‘Collections‘ utilities: sort, min/max, shuffle (overview)
- Wrapped collections: unmodifiable/synchronized/checked (why/when)
- Wrapper classes + autoboxing pitfalls; class loading basics (syllabus)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Collections utilities (common)

- `sort`, `reverse`, `shuffle`

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Collections utilities (common)

- `sort`, `reverse`, `shuffle`
- `min`, `max`, `frequency`

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Collections utilities (common)

- `sort`, `reverse`, `shuffle`
- `min`, `max`, `frequency`
- `binarySearch` (requires sorted list)

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Wrapped collections (why)

- `unmodifiableList`: expose read-only view to callers.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Wrapped collections (why)

- `unmodifiableList`: expose read-only view to callers.
- `synchronizedList`: basic thread-safe wrapper (older style).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Wrapped collections (why)

- `unmodifiableList`: expose read-only view to callers.
- `synchronizedList`: basic thread-safe wrapper (older style).
- `checkedList`: runtime type checking for legacy/raw code.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Wrapper classes + autoboxing pitfalls

- Wrapper classes: `Integer`, `Double`, ...

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Wrapper classes + autoboxing pitfalls

- Wrapper classes: `Integer`, `Double`, ...
- Needed because generics require reference types.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Wrapper classes + autoboxing pitfalls

- Wrapper classes: `Integer`, `Double`, ...
- Needed because generics require reference types.
- Pitfall: unboxing `null` → `NullPointerException`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Wrapper classes + autoboxing pitfalls

- Wrapper classes: `Integer`, `Double`, ...
- Needed because generics require reference types.
- Pitfall: unboxing `null` → `NullPointerException`.
- Pitfall: prefer `equals()` over == for wrappers/strings.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *Wrapped Collections + 'Collections' Utilities + Wrappers* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *`Collections` utilities (common ones)*, *Wrapped collections (why/when)*, the worked example, or the recap?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Example: utilities + wrappers

```
import java.util.*;
public class Demo {
    public static void main(String[] args) {
        List<Integer> a = new ArrayList<>(Arrays.asList(3, 1, 2)
            );
        Collections.sort(a);
        System.out.println("Sorted: " + a);

        List<Integer> ro = Collections.unmodifiableList(a);
        // ro.add(99); // UnsupportedOperationException

        Integer x = null;
        // int y = x; // NullPointerException (unboxing null)
    }
}
```

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. What exception occurs when modifying an unmodifiable list?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. What exception occurs when modifying an unmodifiable list?
2. Why do we need `Integer` instead of `int` in lists?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. What exception occurs when modifying an unmodifiable list?
2. Why do we need `Integer` instead of `int` in lists?
3. What happens when you unbox `Integer x = null`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Watch-outs

- Assuming wrapper objects behave exactly like primitives in every context.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Watch-outs

- Assuming wrapper objects behave exactly like primitives in every context.
- Treating synchronized wrappers as a full concurrency design.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Watch-outs

- Assuming wrapper objects behave exactly like primitives in every context.
- Treating synchronized wrappers as a full concurrency design.
- Forgetting that an unmodifiable view can still reflect external changes to the backing collection.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Takeaways

- `Collections` provides standard algorithms for collections.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Takeaways

- `Collections` provides standard algorithms for collections.
- Wrappers enforce policies (read-only, sync, type checks).

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Key Takeaways

- `Collections` provides standard algorithms for collections.
- Wrappers enforce policies (read-only, sync, type checks).
- Wrapper classes enable generics; watch autoboxing pitfalls.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint Questions

1. Difference: `Collection` vs `Collections`?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint Questions

1. Difference: `Collection` vs `Collections`?
2. When do you use an unmodifiable wrapper?

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Checkpoint Questions

1. Difference: `Collection` vs `Collections`?
2. When do you use an unmodifiable wrapper?
3. Give one autoboxing pitfall.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

Next Unit Preview

Next: Unit 06 – Capstone Project

- We will apply OOP + collections + Swing + JDBC in a complete project.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).